PHASE 1 OF 18

Java Foundations & the JVM

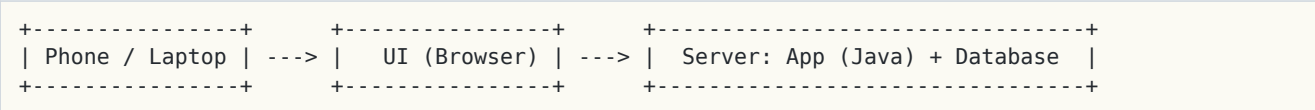
Where Java fits, history and versions, JDK/JRE/JVM, first program, main(), compilation flow and JVM architecture

IN THIS PHASE

1.1 Where Java Fits in Full-Stack Development
1.2 History, Editions and Versions
1.3 Features of Java (WORA: Write Once, Run Anywhere)
1.4 JDK vs JRE vs JVM
1.5 Environment Setup (Windows)
1.6 Your First Java Program
1.7 The main() Method Explained
1.8 Structure of a Java Program
1.9 How a Java Program Runs
1.10 JVM Architecture
1.11 Common Beginner Errors and Fixes
1.12 Interview Questions
1.13 Quick Revision

1.1 Where Java Fits in Full-Stack Development

A full-stack application has a **frontend** (what the user sees and clicks), a **backend** (hidden business logic that bridges the UI and the database), a **database**, and a **server/runtime** that hosts the application. When a user performs an action on the frontend, backend logic runs to handle that operation.



Layer	Responsibility	Common technologies
Frontend	User interface	HTML (structure), CSS, JavaScript (dynamic behaviour), Bootstrap (responsive), React (library), Angular (framework)
Backend	Business logic, APIs, security	**Java (Spring Boot)**, .NET, Python, Node.js, PHP
Database	Persistent data	Oracle, MySQL, PostgreSQL, SQL Server, MongoDB, Cassandra, HBase, Hive
Server / runtime	Runs the application	Tomcat, Jetty, WildFly/JBoss, WebLogic, WebSphere (IIS for .NET)

The Java project tool ecosystem

Category	Tools
Build & dependencies	Maven, Gradle
Version control	Git, GitHub, Bitbucket
Testing	JUnit 5, TestNG, Mockito (mocking), JaCoCo (coverage), JMeter (performance), Postman (API)
Code quality	SonarQube (static analysis / code review)
CI/CD	Jenkins, GitHub Actions (build → test → deploy)
Logging	SLF4J (facade) + Logback or Log4j2
API docs	Swagger / OpenAPI
Containers & orchestration	Docker, Kubernetes
Messaging & caching	Apache Kafka, Redis
Productivity	Project Lombok (removes boilerplate)
Monitoring & ops	Splunk, PuTTY, WinSCP
Project management	Jira (tasks, bugs, sprints)
Cloud	AWS, Azure, GCP, Oracle Cloud, IBM Cloud

1.2 History, Editions and Versions

- Developed at **Sun Microsystems** by a team led by **James Gosling**, starting in **1991** under the name **Oak**.
- Released publicly as **Java in 1995** (JDK 1.0 shipped January 1996).
- **Oracle acquired Sun in 2010**; Oracle now leads Java's development.
- **OpenJDK** is the open-source reference implementation. Free, production-ready OpenJDK builds include Eclipse Temurin, Amazon Corretto, Microsoft Build of OpenJDK and Azul Zulu. Oracle JDK has its own licence terms.
- Official API docs: `https://docs.oracle.com/en/java/javase/` (pick your version).

Editions

Edition	Old name	Used for
Java SE (Standard Edition)	J2SE	Core Java: language, collections, I/O, threads, networking. Base for everything else.
Jakarta EE (Enterprise Edition)	J2EE / Java EE	Enterprise web apps: Servlets, JPA, JAX-RS. Moved to the Eclipse Foundation in 2017; packages renamed `javax.*` → `jakarta.*` from Jakarta EE 9.
Java ME (Micro Edition)	J2ME	Small/embedded devices. Largely legacy.

> **NOTE**
> Android apps can be written in Java, but Android does **not** use Java ME or the standard JVM. It compiles to its own format and runs on the Android Runtime (ART).

Release cadence and LTS versions

Since Java 9 (2017), a new feature release ships **every six months**, and a **Long-Term Support (LTS)** release arrives every two years. Companies standardise on LTS versions.

LTS version	Year	Headline features you should know
Java 8	2014	Lambdas, Streams API, functional interfaces, `Optional`, `java.time`, default/static interface methods

LTS version	Year	Headline features you should know
Java 11	2018	`var` in lambda params, standard `HttpClient`, run single-file source (`java Hello.java`), `String.isBlank()/lines()/strip()/repeat()`
Java 17	2021	Records, sealed classes, pattern matching for `instanceof`, text blocks, switch expressions
Java 21	2023	Virtual threads, pattern matching for `switch`, record patterns, sequenced collections
Java 25	2025	Compact source files & instance `main`, flexible constructor bodies, scoped values, module import declarations

INTERVIEW

Spring Boot 3.x requires **Java 17 or later**. Many enterprise projects (especially in banking) still run Java 8 or 11, so know what changed in each LTS.

1.3 Features of Java (WORA: Write Once, Run Anywhere)

Feature	What it really means
Simple	Familiar C-style syntax, but no pointer arithmetic, no manual memory freeing, no multiple inheritance of classes, no operator overloading.
Platform independent	`javac` produces **bytecode** (`.class`), which runs on any OS that has a JVM.
Object-oriented	Programs are built from classes and objects; supports encapsulation, inheritance, polymorphism and abstraction. Code reuse through inheritance and composition.
Robust	Strong static typing, compile-time checks, exception handling, automatic garbage collection, array bounds checking.
Secure	No direct memory access, bytecode verifier, class-loader isolation, strong typing.
Multithreaded	Built-in threads, `java.util.concurrent`, and virtual threads (Java 21).
Distributed / networked	Built-in networking (`java.net`, `HttpClient`), RMI, used for distributed systems.
Portable	Primitive sizes are fixed (an `int` is always 32-bit) on every platform.
High performance	The JIT compiler turns frequently-run bytecode into native machine code.
Dynamic	Classes are loaded on demand at runtime; reflection lets code inspect classes at runtime.

CORRECTED FROM ORIGINAL NOTES

A few lines in the original notes were inaccurate:

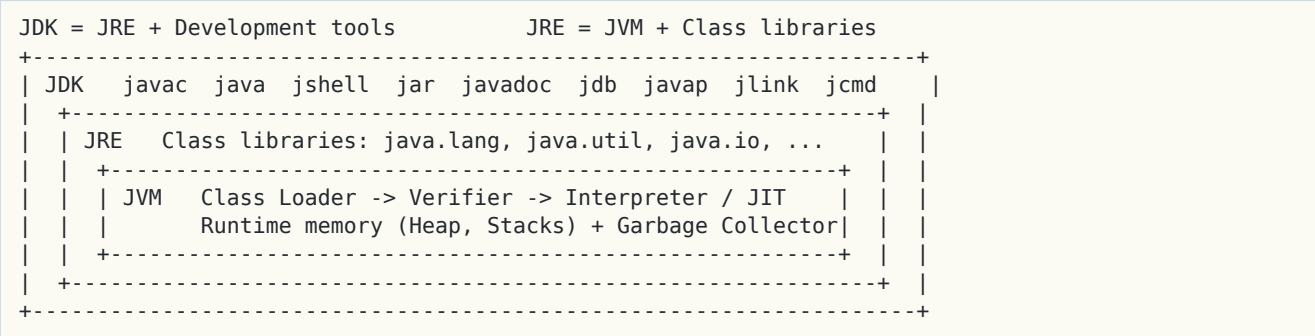
- "Secure: viruses can attack object code but not bytecode": Java's security comes from the verifier, no pointers, sandboxing and class-loader isolation, not from bytecode being immune.
- "Distributed: code need not execute sequentially": that describes **multithreading**. Distributed means built-in support for networked applications.
- "Java is an interpreted language": Java is **both compiled and interpreted**. `javac` compiles to bytecode; the JVM interprets it and JIT-compiles hot code to native code.

INTERVIEW

Is Java 100% object-oriented? **No.** It has primitive types (`int`, `char`, ...) that are not objects, and `static` members that belong to no object. Wrapper classes exist to treat primitives as objects.

1.4 JDK vs JRE vs JVM

- **JVM (Java Virtual Machine):** executes bytecode. Loads classes, verifies them, manages memory (heap, stacks, GC) and runs code via the interpreter and JIT.
- **JRE (Java Runtime Environment):** JVM **plus** the standard class libraries (`java.lang`, `java.util`, `java.io`, ...). Enough to **run** Java programs.
- **JDK (Java Development Kit):** JRE **plus** development tools (`javac`, `jar`, `javadoc`, `jdb`, ...). Needed to **develop** Java programs.



Useful JDK tools

Tool	Purpose
`javac`	Compiles `.java` source into `.class` bytecode
`java`	Launches the JVM to run a class, JAR or (Java 11+) a single source file
`jshell`	Interactive REPL (Java 9+): try snippets without writing a class
`jar`	Packages classes and resources into a `.jar` archive
`javadoc`	Generates HTML API documentation from `/** ... */` comments
`jdb`	Command-line debugger
`javap`	Disassembles a `.class` file (`javap -c` shows bytecode)
`jlink`	Builds a minimal custom runtime image (Java 9+)
`jcmd`, `jstack`, `jmap`, `jconsole`	Diagnostics: thread dumps, heap dumps, monitoring
`keytool`	Manages keystores and certificates (common in banking/TLS setups)

> **NOTE**
> Since **Java 11**, OpenJDK and Oracle no longer ship a separate JRE download. You install the JDK, or build a slim runtime with `jlink`.

> **INTERVIEW**
> Is the JVM platform independent? **No.** Bytecode is platform independent; each OS needs its own JVM implementation. That is what makes the bytecode portable.

1.5 Environment Setup (Windows)

1. Download and install a JDK (for example Eclipse Temurin 21 LTS).
2. Create a system variable `JAVA_HOME` pointing at the JDK folder (not the `bin` folder).
3. Add `%JAVA_HOME%\bin` to the system `Path` so `java` and `javac` work from any folder.
4. Open a **new** Command Prompt and verify:

```
C:\> java -version
openjdk version "21.0.x" ...
C:\> javac -version
javac 21.0.x
C:\> echo %JAVA_HOME%
C:\Program Files\Eclipse Adoptium\jdk-21.0.x-hotspot
```

TIP

Maven, Gradle, IntelliJ and many tools read `JAVA_HOME`, not just `Path`. With multiple JDKs installed, switching `JAVA_HOME` (and reopening the terminal) switches versions.

1.6 Your First Java Program

```java
// File name MUST be Hello.java because the public class is Hello
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

```
javac Hello.java      -> compiles, creates Hello.class
java Hello            -> runs (class name, NOT Hello.class)
java Hello.java       -> Java 11+: compile + run a single file in one step
```

Rules to remember

- Java is **case sensitive**: `Main`, `main` and `MAIN` are different.
- A file can hold **at most one** `public` **top-level class**, and the file name must match it.
- A file may contain several non-public classes; `javac` creates one `.class` per class.
- `java.lang` (`String`, `System`, `Math`, ...) is imported automatically.
- Every statement ends with `;` and blocks are delimited by `{ }`.

`System.out.println()` explained: `System` is a class in `java.lang`, `out` is a static `PrintStream` field of `System`, and `println()` is a method of `PrintStream`. `print()` keeps the cursor on the same line; `println()` moves to the next line.

MODERN JAVA

Java 25 finalised *compact source files and instance main methods*. Beginners can now write a program without declaring a class or `static`:

```java
void main() {
    IO.println("Hello from Java 25!");
}
```

1.7 The main() Method Explained

```java
public static void main(String[] args)
```

Part	Why it is there
`public`	The JVM calls `main` from outside the class, so it must be accessible.
`static`	The JVM calls it **without creating an object** (no instance exists yet).
`void`	It returns nothing to the JVM. Use `System.exit(code)` to return an exit status.
`main`	The method name the JVM looks for as the entry point.
`String[] args`	Command-line arguments, always received as Strings.

Valid variations

```
public static void main(String[] args)       // standard
public static void main(String args[])       // C-style array declaration
public static void main(String... args)      // varargs
static public void main(String[] args)       // modifier order doesn't matter
public static final void main(final String[] args)  // final is allowed
```

INTERVIEW

Common questions about `main`:

- *Can we overload* `main`*?* Yes, but the JVM only calls `main(String[])`.
- *Can we run a class without* `main`*?* Not as a standalone application (static blocks alone stop working from Java 7).
- *What if* `static` *is missing?* Before Java 21 the program compiles but fails at launch ("main method is not static"). Newer launchers can use instance main methods (finalised in Java 25).
- *Is* `args` *ever null?* No. With no arguments, it is an empty array (`args.length == 0`).

1.8 Structure of a Java Program

```
package com.bank.payments;                  // 1. package: optional, at most one, must be first

import java.util.List;                      // 2. imports: optional, zero or more
import java.time.LocalDate;

public class PaymentService {               // 3. type declarations (class/interface/enum/record)

    private static final String CURRENCY = "INR";   // constants
    private double balance;                          // fields (data)

    public PaymentService(double opening) {          // constructor
        this.balance = opening;
    }

    public void pay(double amount) {                 // methods (behaviour)
        balance -= amount;
    }
}
```

Order is fixed: **package → imports → types**. Comments can appear anywhere.

1.9 How a Java Program Runs

```
 Hello.java --javac--> Hello.class (bytecode, platform independent)
                            |
                            v
 +------------------------- JVM ------------------------------+
 | Class Loader  ->  Bytecode Verifier  ->  Execution Engine  |
 |                                          (Interpreter + JIT)|
 +-------------------------------------------------------------+
                            |
                            v
                 Native machine code -> OS + CPU -> Output
```

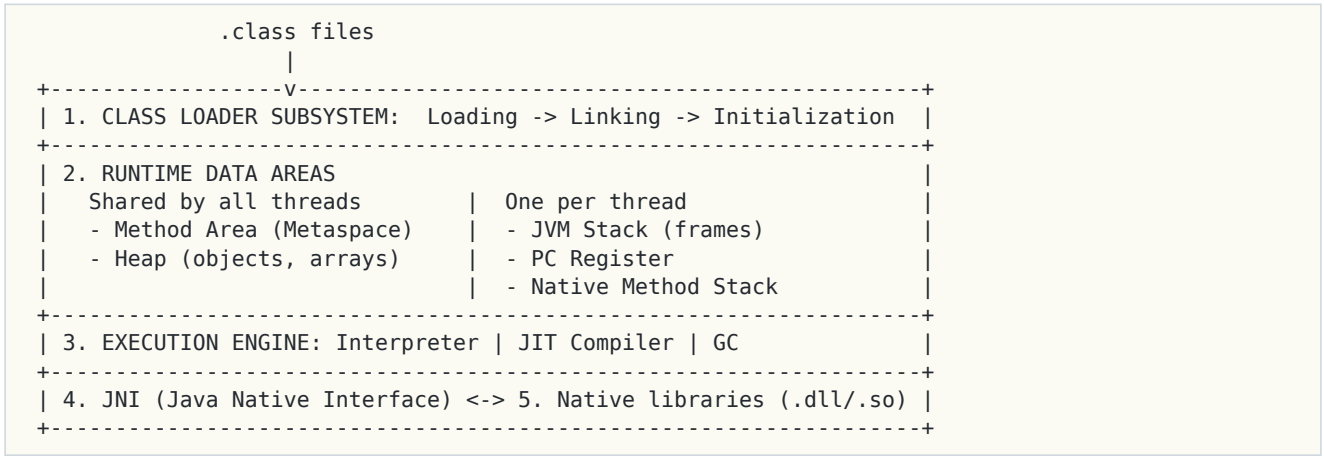
Translators

Translator	How it works	Speed
Interpreter	Translates and runs code line by line	Slower; starts immediately
Compiler	Translates the whole program ahead of time	Fast execution
Assembler	Converts assembly language into machine code	n/a
JIT compiler	Compiles frequently used ("hot") bytecode to native code **at runtime**	Fast after warm-up

Java uses a **compiler** at build time (`javac` → bytecode) and an **interpreter plus JIT** at runtime. The HotSpot JVM starts by interpreting bytecode and counts how often each method runs. Hot methods are compiled to optimised native code (C1 then C2, called *tiered compilation*). That is why Java apps get faster after warm-up.

> **TIP**
> Run `javap -c Hello` to see the actual bytecode instructions inside `Hello.class`.

1.10 JVM Architecture



1. Class Loader Subsystem

Loading reads `.class` files and creates `Class` objects. Three built-in loaders:

Class loader	Loads	Example
Bootstrap	Core platform classes (`java.base` module)	`java.lang.String`, `java.util.List`
Platform (called *Extension* before Java 9)	Other platform modules	`java.sql`, `java.xml`
Application (System)	Your classes and JARs on the classpath	`com.bank.PaymentService`

Loaders follow the **parent delegation model**: a loader first asks its parent to load a class, and only loads it itself if the parent cannot. This stops anyone from replacing core classes such as `java.lang.String` with a fake.

Linking has three steps: **Verify** (bytecode is valid and safe), **Prepare** (memory for static variables, set to default values), and **Resolve** (symbolic references become direct references).

Initialization assigns real values to static variables and runs `static { }` blocks, top to bottom. This happens once, when the class is first used. (Class loading is covered in depth in Phase 15.)

2. Runtime Data Areas

Area	Stores	Scope	Error when exhausted
Method Area (Metaspace since Java 8)	Class metadata, method bytecode, runtime constant pool	Shared	`OutOfMemoryError: Metaspace`
Heap	All objects and arrays, instance variables, String pool (since Java 7)	Shared; managed by GC	`OutOfMemoryError: Java heap space`
JVM Stack	One **frame** per method call: local variables, operand stack, return address	Per thread	`StackOverflowError`
PC Register	Address of the bytecode instruction currently executing	Per thread	n/a
Native Method Stack	State of native (C/C++) method calls	Per thread	`StackOverflowError`

```
class Account {
    static String bank = "ABC Bank";  // static: class-level, one copy per class
    double balance;                   // instance field: lives inside the object (Heap)

    void deposit(double amount) {     // bytecode for this method: Method Area
        double fee = 1.5;             // local variable: in this call's Stack frame
        balance += amount - fee;
    }
}

Account acc = new Account();   // 'acc' reference: Stack | the Account object: Heap
acc.deposit(500);              // new frame pushed on the Stack, popped when it returns
```

> **NOTE**
> Before Java 8, class metadata lived in **PermGen**, which had a fixed size and often caused `OutOfMemoryError: PermGen space`. Java 8 replaced it with **Metaspace**, which uses native memory and grows as needed. Static variables are conceptually class-level; in HotSpot they are physically stored with the `Class` object on the heap.

The heap is divided into **generations**: the Young Generation (Eden, Survivor S0 and S1) for new objects, and the Old Generation for long-lived objects. Garbage collection is covered in Phase 15.

3. Execution Engine

- **Interpreter:** executes bytecode instruction by instruction. Quick to start, slower to run.
- **JIT compiler:** compiles hot methods to native code and caches it (C1 = quick, C2 = highly optimised).
- **Garbage Collector:** automatically frees heap objects that are no longer reachable. **G1** is the default since Java 9; alternatives include ZGC, Shenandoah, Parallel and Serial.

4 & 5. JNI and Native Libraries

The **Java Native Interface** lets Java call C/C++ code and vice versa. Native libraries are `.dll` on Windows, `.so` on Linux and `.dylib` on macOS.

```
public class NativeDemo {
    static { System.loadLibrary("crypto"); }  // loads crypto.dll / libcrypto.so
    public native int encrypt(byte[] data);   // body implemented in C/C++
}
```

Common JVM options

Option	Meaning	Example
`-Xms`	Initial heap size	`-Xms512m`
`-Xmx`	Maximum heap size	`-Xmx2g`
`-Xss`	Stack size per thread	`-Xss1m`
`-XX:MaxMetaspaceSize`	Upper limit for Metaspace	`-XX:MaxMetaspaceSize=256m`
`-XX:+UseG1GC`	Select a garbage collector	`-XX:+UseZGC`
`-Xlog:gc`	Log garbage collection (Java 9+)	`java -Xlog:gc -jar app.jar`
`-XX:+HeapDumpOnOutOfMemoryError`	Write heap dump on OOM	Useful in production

1.11 Common Beginner Errors and Fixes

Error message	Cause	Fix
`'javac' is not recognized ...`	JDK `bin` not on `Path`	Add `%JAVA_HOME%\bin` to `Path`, reopen the terminal
`class Hello is public, should be declared in a file named Hello.java`	File name and public class name differ	Rename the file or the class
`Could not find or load main class Hello`	Wrong name, wrong folder/classpath, or class is in a package	Run from the right folder; for packages use `java com.bank.Hello`
`Main method not found in class Hello`	Wrong `main` signature	Use `public static void main(String[] args)`
`UnsupportedClassVersionError ... class file version 65.0`	Compiled with a newer JDK than the runtime	Run on a newer JDK, or compile with `--release 17`
`StackOverflowError`	Infinite or very deep recursion	Add a base case; refactor to iteration
`OutOfMemoryError: Java heap space`	Too many live objects, or a memory leak	Fix the leak; increase `-Xmx` if truly needed

> **TIP**
> Class-file major versions to recognise: **52** = Java 8, **55** = 11, **61** = 17, **65** = 21, **69** = 25.

1.12 Interview Questions

Q. What is the difference between JDK, JRE and JVM?

The JVM executes bytecode. The JRE is the JVM plus the class libraries needed to run programs. The JDK is the JRE plus development tools such as `javac`, `jar` and `javadoc`. Developers need the JDK; running an app needs only a runtime.

Q. Why is Java platform independent?

`javac` compiles source into bytecode rather than OS-specific machine code. Any OS with a JVM can run that bytecode, and the JVM translates it into native instructions.

Q. Is Java compiled or interpreted?

Both. Source is compiled to bytecode by `javac`. At runtime the JVM interprets bytecode and JIT-compiles hot code into native machine code.

Q. What is the JIT compiler?

The Just-In-Time compiler inside the JVM. It detects frequently executed methods and compiles them to optimised native code at runtime, improving performance after warm-up.

Q. What are the JVM memory areas?

Method Area/Metaspace and Heap (shared by all threads), plus the JVM Stack, PC Register and Native Method Stack (one per thread).

Q. What is the difference between Stack and Heap memory?

The stack holds method frames (local variables and references), is per thread, and is freed automatically when a method returns. The heap holds all objects, is shared by all threads, and is cleaned by the garbage collector. Stack overflow gives `StackOverflowError`; heap exhaustion gives `OutOfMemoryError`.

Q. What is the parent delegation model?

A class loader first delegates loading to its parent (Application → Platform → Bootstrap). It loads the class itself only if the parent cannot. This prevents duplicate loading and protects core classes from being replaced.

Q. What replaced PermGen in Java 8, and why?

Metaspace. PermGen had a fixed maximum size and caused frequent out-of-memory errors. Metaspace uses native memory and can grow dynamically (cap it with `-XX:MaxMetaspaceSize`).

Q. Why is `main` static?

The JVM must call it before any object of the class exists. A static method can be invoked using just the class name.

Q. What happens when you run `java Hello`?

The JVM starts, the Application class loader loads `Hello.class` (delegating to its parents), the class is verified, prepared and initialized (static blocks run), and then the execution engine invokes `main` on the main thread.

1.13 Quick Revision

- Java: Sun Microsystems, James Gosling, 1991 (Oak), released 1995; owned by Oracle since 2010.
- WORA: source → `javac` → bytecode → any OS's JVM.
- LTS versions: **8, 11, 17, 21, 25**. Spring Boot 3 needs Java 17+.
- **JDK = JRE + tools; JRE = JVM + libraries.** The JVM is platform dependent; bytecode is not.
- Java is **compiled and interpreted**; the JIT makes hot code native.
- `main` is `public static void main(String[] args)`; one public class per file, matching the file name.
- JVM = Class Loader (Load → Link → Initialize) + Runtime Data Areas + Execution Engine + JNI.
- Shared memory: **Heap** and **Metaspace**. Per thread: **Stack**, **PC Register**, **Native Stack**.
- Class loaders: Bootstrap → Platform → Application, using parent delegation.
- GC cleans the heap; G1 is the default collector since Java 9.